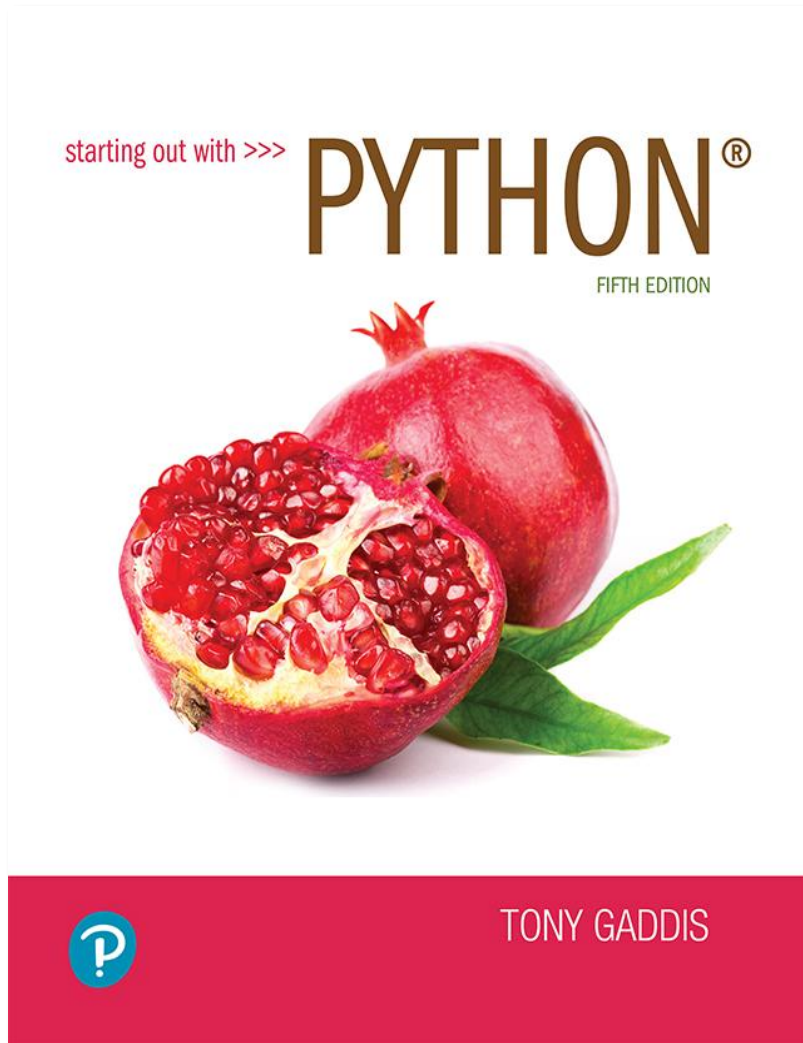


Starting out with Python

Fifth Edition



Chapter 7

Lists and Tuples

Topics (1 of 2)

- **Sequences**
- **Introduction to Lists**
- **List Slicing**
- **Finding Items in Lists with the in Operator**
- **List Methods and Useful Built-in Functions**

Topics (2 of 2)

- **Copying Lists**
- **Processing Lists**
- **List Comprehensions**
- **Two-Dimensional Lists**
- **Tuples**
- **Plotting List Data with the `matplotlib` Package**

Introduction

- **Have you ever used a list in your everyday life?**
 - Grocery list
 - To-do list
 - List of phone numbers

Introduction

- **Why Need Lists / Tuples?**
 - Limitations of using individual variables to store related data.
 - Ex: keep track of friends names (freind1, freind2,)
 - Lists/Tuples allow **grouping related data together in one variable**, making it easier to manage.
 - Lists/Tuples are a **way to organize** data in Python (Similar to **Arrays** in other languages)

Sequences

- **Sequence**: an object that contains **multiple items** of data
 - The items are stored in **sequence** one after another
- Python provides different types of sequences, including **lists** and **tuples**
 - The difference between these is that a list is **mutable** and a tuple is **immutable**

Introduction to Lists (1 of 2)

- **List:** an object that contains multiple data items
 - Element: An item in a list
 - **Format:** *list = [item1, item2, etc.]*
 - Can hold items of different types
- **print** function can be used to display an entire list
- **list()** function can convert certain types of objects to lists
- **range()** function can be used with **list()**

Introduction to Lists (cont'd.)

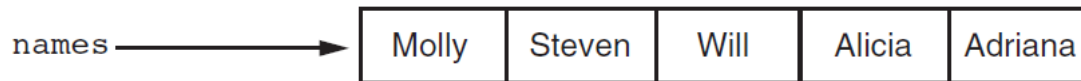
```
even_numbers = [2, 4, 6, 8, 10]
```

Figure 7-1 A list of integers



```
names = ['Molly', 'Steven', 'Will', 'Alicia', 'Adriana']
```

Figure 7-2 A list of strings



```
info = ['Alicia', 27, 1550.87]
```

Figure 7-3 A list holding different types



```
numbers = list(range(5))
```

```
numbers = list(range(1, 10, 2))
```


The Repetition Operator and Iterating over a List

- **Repetition operator**: makes multiple copies of a list and joins them together
 - The ***** symbol is a repetition operator when applied to a sequence and an integer
 - Sequence is left operand, number is right
 - General format: ***list * n*** **=> *[0]*5***
- You can iterate over a list using a **for** loop
 - **Format: *for x in list:***

Indexing

- **Index**: a number specifying the position of an element in a list
 - Enables access to individual element in list
 - Index of **first** element in the list is **0**, **second** element is **1**, and n'th element is $n-1$
 - **Negative indexes** identify positions relative to the end of the list
 - The index -1 identifies the last element, -2 identifies the next to last element, etc.

Indexing

```
my_list = [10, 20, 30, 40]
```

```
print(my_list[0], my_list[1], my_list[2], my_list[3])
```

```
index = 0
while index < 4:
    print(my_list[index])
    index += 1
```

```
print(my_list[-1], my_list[-2], my_list[-3], my_list[-4])
```

- An **IndexError** exception is raised if an invalid index is used

The len function

- len function: returns the length of a sequence such as a list
 - Example: `size = len(my_list)`
 - Returns the number of elements in the list, so the index of **last element** is `len(list) - 1`
 - Can be used to **prevent** an **IndexError** exception when iterating over a list with a loop

```
my_list = [10, 20, 30, 40]
index = 0
while index < len(my_list):
    print(my_list[index])
    index += 1
```

Using for Loop and Index

```
1 names = ['Jenny', 'Kelly', 'Chloe', 'Aubrey']  
2 for index in range(len(names)):  
3     print(names[index])
```

Lists Are Mutable

- **Mutable sequence**: the items in the sequence can be changed
 - Lists are mutable, and so their elements can be changed
- An expression such as
- `list[1] = new_value` can be used to assign a new value to a list element
 - Must use a **valid index** to prevent raising of an `IndexError` exception

Example

```
sales = [0]*7
print("Enter sales for each day: ")
for index in range(len(sales)):
    sales[index] = float(input("Enter sales: "))

for value in sales:
    print(value)
```

Concatenating Lists

- **Concatenate**: join two things together
- The **+** operator can be used to concatenate two lists
 - **Cannot** concatenate a list with another data type, such as a number
- The **+=** augmented assignment operator can also be used to concatenate lists

Examples

```
list1 = [1, 2, 3, 4]  
list2 = [5, 6, 7, 8]  
list3 = list1 + list2
```

```
list1 = [1, 2, 3, 4]  
list2 = [5, 6, 7, 8]  
list1 += list2
```

```
>>> girl_names = ['Joanne', 'Karen', 'Lori'] Enter  
>>> boy_names = ['Chris', 'Jerry', 'Will'] Enter  
>>> all_names = girl_names + boy_names Enter  
>>> print(all_names) Enter  
['Joanne', 'Karen', 'Lori', 'Chris', 'Jerry', 'Will']
```



Test your understanding

Checkpoint

7.1 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]
numbers[2] = 99
print(numbers)
```

7.2 What will the following code display?

```
numbers = list(range(3))
print(numbers)
```

7.3 What will the following code display?

```
numbers = [10] * 5
print(numbers)
```



Test your understanding

7.4 What will the following code display?

```
numbers = list(range(1, 10, 2))  
for n in numbers:  
    print(n)
```

7.5 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]  
print(numbers[-2])
```

7.6 How do you find the number of elements in a list?



Test your understanding

7.7 What will the following code display?

```
numbers1 = [1, 2, 3]
numbers2 = [10, 20, 30]
numbers3 = numbers1 + numbers2
print(numbers1)
print(numbers2)
print(numbers3)
```

7.8 What will the following code display?

```
numbers1 = [1, 2, 3]
numbers2 = [10, 20, 30]
numbers2 += numbers1
print(numbers1)
print(numbers2)
```

List Slicing

- **Slice**: a span of items that are taken from a sequence
 - **List slicing format:** *list[start : end]*
 - Span is a list containing copies of elements from *start* up to, but not including, *end*
 - If *start* not specified, 0 is used for start index
 - If *end* not specified, `len(list)` is used for end index
 - Slicing expressions can include a **step value** and **negative indexes** relative to end of list

List Slicing

```
days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',  
        'Thursday', 'Friday', 'Saturday']
```

```
mid_days = days[2:5]
```

```
['Tuesday', 'Wednesday', 'Thursday']
```

```
numbers = [1, 2, 3, 4, 5]
```

```
>>> print(numbers[2:])  
[3, 4, 5]
```

```
>>> print(numbers[:3])  
[1, 2, 3]
```

```
>>> print(numbers[:])  
[1, 2, 3, 4, 5]
```

```
>>> print(numbers[1:3])  
[2, 3]
```

```
>>> print(numbers[1:8:2])  
[2, 4, 6, 8]
```



Test your understanding

Checkpoint

7.9 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]
my_list = numbers[1:3]
print(my_list)
```

7.10 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]
my_list = numbers[1:]
print(my_list)
```

7.11 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]
my_list = numbers[:1]
print(my_list)
```



Test your understanding

7.12 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]
my_list = numbers[:]
print(my_list)
```

7.13 What will the following code display?

```
numbers = [1, 2, 3, 4, 5]
my_list = numbers[-3:]
print(my_list)
```


Finding Items in Lists with the `in` Operator

- You can use the `in` operator to determine whether an item is contained in a list
 - General format: `item in list`
 - Returns `True` if the item is in the list, or `False` if it is not in the list
- Similarly you can use the `not in` operator to determine whether an item is not in a list

Finding Items in Lists with the `in` Operator

```
def main():  
    # Create a list of product numbers.  
    prod_nums = ['V475', 'F987', 'Q143', 'R688']  
  
    # Get a product number to search for.  
    search = input('Enter a product number: ')  
  
    # Determine whether the product number is in the list.  
    if search in prod_nums:  
        print(search, 'was found in the list.')  
    else:  
        print(search, 'was not found in the list.')  
  
# Call the main function.  
main()
```

```
if search not in prod_nums:  
    print(search, 'was not found in the list.')  
else:  
    print(search, 'was found in the list.')  

```



Test your understanding

Checkpoint

7.14 What will the following code display?

```
names = ['Jim', 'Jill', 'John', 'Jasmine']  
if 'Jasmine' not in names:  
    print('Cannot find Jasmine.')  
else:  
    print("Jasmine's family:")  
    print(names)
```

List Methods and Useful Built-in Functions (1 of 4)

- **`append(item)`**: used to add items to a list
 - *item* is appended to the end of the existing list
- **`index(item)`**: used to determine where an item is located in a list
 - Returns the index of the first element in the list containing *item*
 - Raises **`ValueError`** exception if *item* not in the list

List Methods and Useful Built-in Functions (2 of 4)

- **insert(*index*, *item*)**: used to insert *item* at position *index* in the list
- **sort()**: used to sort the elements of the list in ascending order
- **remove(*item*)**: removes the first occurrence of *item* in the list
- **reverse()**: reverses the order of the elements in the list

List Methods and Useful Built-in Functions (3 of 4)

Table 7-1 A few of the list methods

Method	Description
<code>append(item)</code>	Adds <i>item</i> to the end of the list.
<code>index(item)</code>	Returns the index of the first element whose value is equal to <i>item</i> . A <code>ValueError</code> exception is raised if <i>item</i> is not found in the list.
<code>insert(index, item)</code>	Inserts <i>item</i> into the list at the specified <i>index</i> . When an item is inserted into a list, the list is expanded in size to accommodate the new item. The item that was previously at the specified index, and all the items after it, are shifted by one position toward the end of the list. No exceptions will occur if you specify an invalid index. If you specify an index beyond the end of the list, the item will be added to the end of the list. If you use a negative index that specifies an invalid position, the item will be inserted at the beginning of the list.
<code>sort()</code>	Sorts the items in the list so they appear in ascending order (from the lowest value to the highest value).
<code>remove(item)</code>	Removes the first occurrence of <i>item</i> from the list. A <code>ValueError</code> exception is raised if <i>item</i> is not found in the list.
<code>reverse()</code>	Reverses the order of the items in the list.

List Methods and Useful Built-in Functions: **Append**

```
def main():  
    names=[]  
    again="y"  
    while again=='y':  
        name=input("Enter a name:")  
        names.append(name)  
        print("Do you want an other name?")  
        again=input("y:yes, any other thing:no")  
        print()  
    print("Here are the list of the names:")  
    for n in names:  
        print(n)
```

List Methods and Useful Built-in Functions: **Index, Insert**

```
def main():  
    classes = ["CA221", "CA222", "CNS222", "CNS224"]  
    print(classes)  
    name = input("Enter class name to shift: ")  
    position = classes.index(name)  
    new_class = input("Enter new class to insert: ")  
    classes.insert(position, new_class)  
    print(classes)
```


List Methods and Useful Built-in Functions: **Sort, Reverse**

```
my_list = [9, 1, 0, 2, 8, 6, 7, 4, 5, 3]
print('Original order:', my_list)
my_list.sort()
print('Sorted order:', my_list)
```

```
my_list = [1, 2, 3, 4, 5]
print('Original order:', my_list)
my_list.reverse()
print('Reversed:', my_list)
```

List Methods and Useful Built-in Functions: **Remove**

```
def main():  
    classes = ["CA221", "CA222", "CNS222", "CNS224"]  
    print(classes)  
    name = input("Enter class to remove: ")  
    if name in classes:  
        classes.remove(name)  
    else:  
        print("Class name not found!!")  
    print(classes)
```

List Methods and Useful Built-in Functions (4 of 4)

- **del** **statement**: removes an element from a specific index in a list
 - General format: **del *list*[*i*]**
- **min** and **max** **functions**: built-in functions that returns the item that has the lowest or highest value in a sequence
 - The sequence is passed as an argument

List Methods and Useful Built-in Functions: **Del, Min, Max**

```
my_list = [1, 2, 3, 4, 5]
print('Before deletion:', my_list)
del my_list[2]
print('After deletion:', my_list)
```

```
Before deletion: [1, 2, 3, 4, 5]
After deletion: [1, 2, 4, 5]
```

```
my_list = [5, 4, 3, 2, 50, 40, 30]
print('The lowest value is', min(my_list))
```

```
my_list = [5, 4, 3, 2, 50, 40, 30]
print('The highest value is', max(my_list))
```



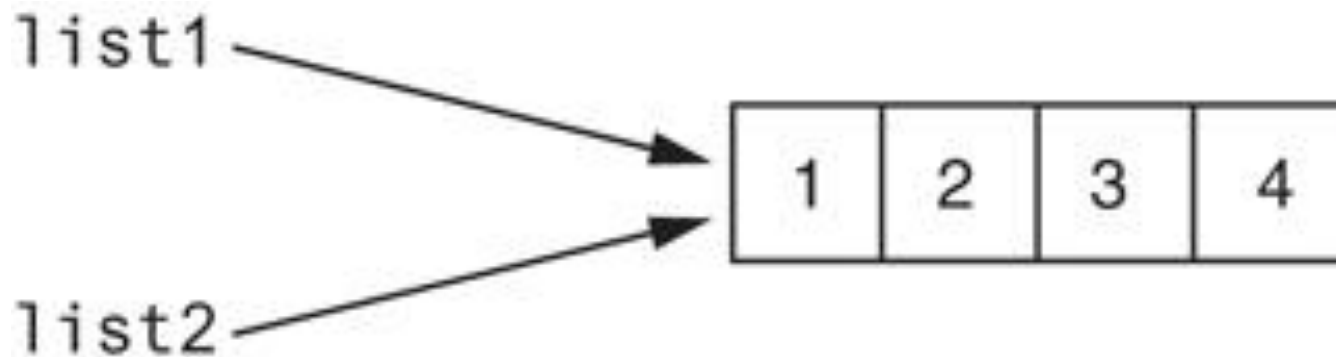
Test your understanding

Checkpoint

- 7.15 What is the difference between calling a list's `remove` method and using the `del` statement to remove an element?
- 7.16 How do you find the lowest and highest values in a list?
- 7.17 Assume the following statement appears in a program:
- ```
names = []
```
- Which of the following statements would you use to add the string 'Wendy' to the list at index 0? Why would you select this statement instead of the other?
- a. `names[0] = 'Wendy'`
  - b. `names.append('Wendy')`
- 7.18 Describe the following list methods:
- a. `index`
  - b. `insert`
  - c. `sort`
  - d. `reverse`

# Copying Lists (2 of 2)

```
list1 = [1, 2, 3, 4]
Assign the list to the list2 variable.
list2 = list1
```



**Figure 7-5** `list1` and `list2` reference the same list

## Copying Lists (cont'd.)

```
>>> list1 = [1, 2, 3, 4]
>>> list2 = list1 Enter
>>> print(list1) Enter
[1, 2, 3, 4]
>>> print(list2) Enter
[1, 2, 3, 4]
>>> list1[0] = 99 Enter
>>> print(list1) Enter
[99, 2, 3, 4]
>>> print(list2) Enter
[99, 2, 3, 4]
```

# Copying Lists (1 of 2)

- To make a copy of a list you must copy each element of the list
  - Two methods to do this:
    - Creating a **new empty list** and **using a for loop** to add a copy of each element from the original list to the new list
    - Creating a **new empty list** and **concatenating** the old list to the new empty list



## Copying Lists (cont'd.)

```
list1 = [1, 2, 3, 4]
Create an empty list.
list2 = []
Copy the elements of list1 to list2.
for item in list1:
 list2.append(item)
```

## Copying Lists (cont'd.)

```
Create a list with values.
list1 = [1, 2, 3, 4]
Create a copy of list1.
list2 = [] + list1
```

## Processing Lists (1 of 2)

# Spotlight

# Page 384

# Processing Lists (1 of 2)

```
NUM_EMPLOYEES = 3

def main():
 employees = [0] * NUM_EMPLOYEES
 for index in range(len(employees)):
 employees[index] = int(input(f"Enter hours for {index+1}: "))
 pay_rate = float(input("Enter pay rate: "))
 for index in range(len(employees)):
 gross_pay = employees[index] * pay_rate
 print(f"gross pay for employee {index+1}= ${gross_pay:.2f}")
```

# Processing Lists (1 of 2)

- List elements can be used in calculations
- To **calculate total** of numeric values in a list use loop with accumulator variable
- To **average** numeric values in a list:
  - Calculate total of the values
  - Divide total of the values by `len(list)`

# Processing Lists

## Ex: Totaling the value in a List

```
def ex2(): #Totaling a value in a list
 numbers=[1,2,3,4,5]
 total=0
 for n in numbers:
 total+=n
 print("total values:", total)
```

# Processing Lists

## Ex: Averaging the value in a List

```
def ex3(): #averaging the values in alist
 numbers=[1,2,2,1,4,5,3]
 total=0
 for n in numbers:
 total+=n
 average= total / len(numbers)
 print("The average is:", format(average, ',.2f'))
```

# Processing Lists

## Ex: Passing list as an argument to a function

```
def ex4(): #Passing a list as an argument to a function
 numbers=[1,2,4,3,2,5]
 print("Total:", get_total(numbers))

def get_total(numbers):
 total=0
 for n in numbers:
 total+=n
 return total
```



# Processing Lists

## Ex: Returning a list from a function

```
def ex5(): #returning a list from a function
 numbers=[]
 for x in range(10):
 print("Enter a value:",end=' ')
 number = int(input())
 numbers.append(number)
 return numbers
```

## Processing Lists (1 of 2)

# Exercise Spotlight

Page 390

# List Comprehensions (1 of 7)

- List comprehension: a **concise expression** that **creates a new** list by iterating over the elements of an **existing** list.

# List Comprehensions (2 of 7)

- The following code uses a for loop to make a copy of a list:

```
list1 = [1, 2, 3, 4]
list2 = []

for item in list1:
 list2.append(item)
```

- The following code uses a **list comprehension** to make a copy of a list:

```
list1 = [1, 2, 3, 4]
list2 = [item for item in list1]
```

# List Comprehensions (3 of 7)

```
list2 = [item for item in list1]
```

Result Expression      Iteration Expression

- The iteration expression works like a for loop
- In this example, it iterates over the elements of `list1`
- Each time it iterates, the target variable `item` is assigned the value of an element.
- At the end of each iteration, the value of the result expression is appended to the new list.

# List Comprehensions (4 of 7)

```
list1 = [1, 2, 3, 4]
list2 = []

for item in list1:
 list2.append(item**2)
```

```
list1 = [1, 2, 3, 4]
list2 = [item**2 for item in list1]
```

- After this code executes, `list2` will contain the values `[1, 4, 9, 16]`

# List Comprehensions (5 of 7)

```
str_list = ['Winken', 'Blinken', 'Nod']
len_list = []

for s in str_list:
 len_list.append(len(s))
```

```
str_list = ['Winken', 'Blinken', 'Nod']
len_list = [len(s) for s in str_list]
```

- After this code executes, `len_list` will contain the values `[6, 7, 3]`

# List Comprehensions (6 of 7)

- You can use an **if clause** in a list comprehension to select only certain elements when processing a list

```
list1 = [1, 12, 2, 20, 3, 15, 4]
list2 = []
```

```
for n in list1:
 if n < 10:
 list2.append(n)
```

Works the same as...

```
list1 = [1, 12, 2, 20, 3, 15, 4]
list2 = [item for item in list1 if item < 10]
```



# List Comprehensions (7 of 7)

```
list1 = [1, 12, 2, 20, 3, 15, 4]
list2 = []
```

```
for n in list1:
 if n < 10:
 list2.append(n)
```

```
list1 = [1, 12, 2, 20, 3, 15, 4]
list2 = [item for item in list1 if item < 10]
```

- After this code executes, list2 will contain [1, 2, 3, 4]



# Test your understanding

## Checkpoint

7.19 Look at the following list comprehension:

```
[x for x in my_list]
```

What is the result expression? What is the iteration expression?

7.20 After this code executes, what value will the `list2` list hold?

```
list1 = [1, 12, 2, 20, 3, 15, 4]
```

```
list2 = [n*2 for n in list1]
```

7.21 After this code executes, what value will the `list2` list hold?

```
list1 = [1, 12, 2, 20, 3, 15, 4]
```

```
list2 = [n for n in list1 if n > 10]
```

# Two-Dimensional Lists (1 of 3)

- **Two-dimensional list:** a list that contains other lists as its elements
  - Also known as **nested list**
  - Common to think of two-dimensional lists as having **rows** and **columns**
  - Useful for working with multiple sets of data
- To process data in a two-dimensional list need to use **two indexes**
- Typically use nested loops to process

# Two-Dimensional Lists (2 of 3)

```
[['Joe', 'Kim'], ['Sam', 'Sue'], ['Kelly', 'Chris']]
```

|       | Column 0 | Column 1 |
|-------|----------|----------|
| Row 0 | 'Joe'    | 'Kim'    |
| Row 1 | 'Sam'    | 'Sue'    |
| Row 2 | 'Kelly'  | 'Chris'  |

**Figure 7-8** A two-dimensional list

# Two-Dimensional Lists (2 of 3)

|       | Column 0 | Column 1 |
|-------|----------|----------|
| Row 0 | 'Joe'    | 'Kim'    |
| Row 1 | 'Sam'    | 'Sue'    |
| Row 2 | 'Kelly'  | 'Chris'  |

```
[['Joe', 'Kim'], ['Sam', 'Sue'], ['Kelly', 'Chris']]
>>> print(students[0]) Enter
['Joe', 'Kim']
>>> print(students[1]) Enter
['Sam', 'Sue']
>>> print(students[2]) Enter
['Kelly', 'Chris']
```

**Figure 7-8** A two-dimensional list

# Two-Dimensional Lists (2 of 3)

```
scores = [[0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]
```

The elements in row 0 are referenced as follows:

```
scores[0][0]
scores[0][1]
scores[0][2]
```

The elements in row 1 are referenced as follows:

```
scores[1][0]
scores[1][1]
scores[1][2]
```

And, the elements in row 2 are referenced as follows:

```
scores[2][0]
scores[2][1]
scores[2][2]
```

**Figure 7-8**

# Two-Dimensional Lists (3 of 3)

|       | Column 0                  | Column 1                  | Column 2                  |
|-------|---------------------------|---------------------------|---------------------------|
| Row 0 | <code>scores[0][0]</code> | <code>scores[0][1]</code> | <code>scores[0][2]</code> |
| Row 1 | <code>scores[1][0]</code> | <code>scores[1][1]</code> | <code>scores[1][2]</code> |
| Row 2 | <code>scores[2][0]</code> | <code>scores[2][1]</code> | <code>scores[2][2]</code> |

**Figure 7-10** Subscripts for each element of the scores list

# Two-Dimensional Lists (3 of 3)

## Program 7-18 (two\_dimensional\_list.py)

```
1 # This program demonstrates a two-dimensional list.
2
3 def main():
4 # Create a two-dimensional list.
5 values = [[1, 2, 3],
6 [10, 20, 30],
7 [100, 200, 300]]
8
9 # Display the list elements.
10 for row in values:
11 for element in row:
12 print(element)
```

### Program Output

```
1
2
3
10
20
30
100
200
300
```

**Figure 7-10** Subscripts for each element of the scores list



## Program 7-19 (random\_numbers.py)

```
1 # This program assigns random numbers to
2 # a two-dimensional list.
3 import random
4
5 # Constants for rows and columns
6 ROWS = 3
7 COLS = 4
8
9 def main():
10 # Create a two-dimensional list.
11 values = [[0, 0, 0, 0],
12 [0, 0, 0, 0],
13 [0, 0, 0, 0]]
14
15 # Fill the list with random numbers.
16 for r in range(ROWS):
17 for c in range(COLS):
18 values[r][c] = random.randint(1, 100)
19
20 # Display the random numbers.
21 print(values)
22
```

### Program Output

```
[[4, 17, 34, 24], [46, 21, 54, 10], [54, 92, 20, 100]]
```



# Test your understanding



## Checkpoint

- 7.22 Look at the following interactive session, in which a two-dimensional list is created. How many rows and how many columns are in the list?

```
numbers = [[1, 2], [10, 20], [100, 200], [1000, 2000]]
```

- 7.23 Write a statement that creates a two-dimensional list with three rows and four columns. Each element should be assigned the value 0.
- 7.24 Write a set of nested loops that display the contents of the `numbers` list shown in Checkpoint question 7.22.

# Tuples (1 of 3)

- **Tuple**: an immutable sequence
  - Very **similar** to a list
  - Once it is created it **cannot be changed**
  - **Format: tuple\_name = (item1, item2)**
  - Tuples **support** operations as lists
    - Subscript indexing for retrieving elements
    - Methods such as **index**
    - Built in functions such as **len, min, max**
    - **Slicing** expressions
    - The **in, +, and \*** operators

# Tuples (2 of 3)

- Tuples **do not support** the methods:
  - **append**
  - **remove**
  - **insert**
  - **reverse**
  - **sort**

# Tuples (3 of 3)

- **Advantages for using tuples over lists:**
  - Processing tuples is **faster** than processing lists
  - Tuples are **safe**
  - Some operations in Python **require** use of tuples
- **list() function:** converts tuple to list
- **tuple() function:** converts list to tuple



# Test your understanding



## Checkpoint

- 7.25 What is the primary difference between a list and a tuple?
- 7.26 Give two reasons why tuples exist.
- 7.27 Assume `my_list` references a list. Write a statement that converts it to a tuple.
- 7.28 Assume `my_tuple` references a tuple. Write a statement that converts it to a list.



# Test your understanding

## 3. Rainfall Statistics

Design a program that lets the user enter the total rainfall for each of 12 months into a list. The program should calculate and display the total rainfall for the year, the average monthly rainfall, the months with the highest and lowest amounts.

# Summary

- This chapter covered:
  - Lists, including:
    - Repetition and concatenation operators
    - Indexing
    - Techniques for processing lists
    - Slicing and copying lists
    - List methods and built-in functions for lists
    - Two-dimensional lists
  - Tuples, including:
    - Immutability
    - Difference from and advantages over lists